

Arquitectura de Computadoras
(CS3051)

Diseño e implementación de un procesador RISC-V superescalar Dual-Issue con optimización microarquitectónica

Proyecto 2 - Primera entrega

Alvarado León, Adriana Celeste
Daza Yalta, Gino Jesús
Ccolcca Flores, Marylin Regina

Profesor: Williams Valencia , Carlos Eduardo
4 de noviembre del 2026

Índice

1. Introducción **3**
1.1. Pipeline de Cinco Etapas . . . 3
1.2. Datapath . . . 3
1.3. Unidad de Control . . . 4

2. Hazard Unit y Datapath Single-Issue **4**
2.1. Cambios e Implementación de la Hazard Unit . . . 4
2.2. Diagrama del Datapath Single-Issue . . . 5

3. Waveforms de los Cuatro Programas **7**
3.1. Programa 1 . . . 7
3.2. Programa 2 . . . 8
3.3. Programa 3 . . . 9
3.4. Programa 4 . . . 11
3.5. Señales relevantes . . . 13
3.6. Resultados intermedios . . . 13

4. Implementación Preliminar del Dual-Issue **14**
4.1. Cambios en Fetch . . . 14
4.2. Cambios en Decode . . . 14
4.3. Módulos incorporados . . . 15
4.4. Datapath Dual-Issue Preliminar . . . 15
4.5. Evidencia del Dual Fetch y Dual Decode . . . 16

1. Introducción

El presente proyecto tiene como objetivo diseñar e implementar un procesador RISC-V de 32 bits con arquitectura superescalar Dual-Issue, que sea capaz de emitir hasta dos instrucciones por ciclo. Como base, se desarrolla un procesador Pipeline Single-Issue de cinco etapas en Verilog, incorporando mecanismos de *forwarding*, *stalling* y *flushing* para resolver dependencias de datos y riesgos de control.

En esta primera entrega, verificamos el funcionamiento del pipeline mediante programas de prueba e iniciamos su adaptación hacia una arquitectura Dual-Issue.

1.1. Pipeline de Cinco Etapas

El procesador implementa una arquitectura segmentada en cinco etapas, permitiendo que varias instrucciones se procesen simultáneamente en diferentes fases:

- **Instruction Fetch (IF):** obtiene la instrucción desde memoria utilizando el Program Counter (PC). Normalmente, el PC avanza cuatro bytes, aunque puede actualizarse ante un salto o detenerse mediante `StallF`.
- **Instruction Decode (ID):** decodifica la instrucción, identifica los registros fuente y destino, lee los operandos del Register File y genera las señales de control.
- **Execute (EX):** ejecuta operaciones aritméticas y lógicas mediante la ALU, selecciona operandos y calcula las direcciones y condiciones de salto.
- **Memory Access (MEM):** realiza las operaciones de lectura y escritura sobre la memoria de datos, utilizando la dirección calculada en EX.
- **Write Back (WB):** selecciona y escribe en el Register File el resultado proveniente de la ALU, la memoria o PC+4.

Las etapas están conectadas mediante cuatro registros de pipeline: IF/ID, ID/EX, EX/MEM y MEM/WB. Estos conservan los datos y las señales de control correspondientes a cada instrucción. El procesador admite como máximo una nueva instrucción por ciclo, salvo cuando se requieren detenciones o anulaciones.

1.2. Datapath

El datapath, implementado en `datapath.v`, integra los registros de pipeline, el Register File, la ALU, los sumadores y los multiplexores necesarios para ejecutar las instrucciones.

En IF, el PC determina la dirección de lectura y se calcula PC+4. En ID, se identifican los registros fuente (`Rs1D`, `Rs2D`) y destino (`RdD`), se leen los operandos y se extiende el inmediato. El Register File dispone de dos puertos de lectura combinacional y uno de escritura en el flanco de bajada del reloj.

En EX, los multiplexores controlados por `ForwardAE` y `ForwardBE` seleccionan los operandos del Register File o los resultados adelantados desde MEM y WB. La señal `ALUSrcE` permite seleccionar un inmediato como segundo operando de la ALU.

Posteriormente, el resultado se transfiere a MEM para realizar los accesos correspondientes. En WB, `ResultSrcW` selecciona el dato que se escribirá en el registro destino. El

datapath también incorpora la actualización del PC mediante `PCSrcE` y las conexiones necesarias para el control de hazards.

1.3. Unidad de Control

La unidad de control, implementada en `controller.v`, interpreta las instrucciones y genera las señales que coordinan el datapath. Está compuesta principalmente por los módulos `maindec.v` y `aludec.v`.

El módulo `maindec` decodifica el *opcode* y genera señales como `RegWrite`, `MemWrite`, `ALUSrc`, `ResultSrc`, `ImmSrc`, `Branch` y `Jump`. Por su parte, `aludec` determina la operación de la ALU mediante `ALUOp`, `funct3` y `funct7`, generando `ALUControl`.

Las señales de control avanzan junto con cada instrucción mediante registros de pipeline, actuando en las etapas correspondientes. La resolución de saltos se realiza en EX mediante:

$$PCSrcE = (BranchE \wedge ZeroE) \vee JumpE \tag{1}$$

Asimismo, la unidad recibe `FlushE` desde la Hazard Unit para anular las señales de control de instrucciones que deben descartarse.

Esta organización modular permite coordinar la ejecución del procesador Single-Issue y constituye la base para su posterior extensión Dual-Issue.

2. Hazard Unit y Datapath Single-Issue

La arquitectura Pipeline Single-Issue incorpora una Hazard Unit, implementada en `hazard.v`, para gestionar dependencias de datos y riesgos de control. Esta unidad se conecta con el datapath y el controlador mediante `riscv.v`, implementando tres mecanismos principales: *forwarding*, *stalling* y *flushing*.

2.1. Cambios e Implementación de la Hazard Unit

La incorporación de la Hazard Unit requirió añadir multiplexores de forwarding y registros de pipeline con señales de habilitación y limpieza. Su lógica combinacional compara registros fuente y destino de distintas etapas y genera las señales `ForwardAE`, `ForwardBE`, `StallF`, `StallD`, `FlushD` y `FlushE`.

a) Forwarding El forwarding permite resolver dependencias RAW (*Read After Write*) utilizando resultados de instrucciones anteriores sin esperar su escritura en el Register File.

La unidad compara los registros fuente en Execute (`Rs1E`, `Rs2E`) con los destinos en Memory y Write Back (`RdM`, `RdW`). Las señales `ForwardAE` y `ForwardBE` controlan los multiplexores de operandos según la siguiente codificación:

Selección	Origen del operando
00	Register File
01	Write Back (`ResultW`)
10	Memory (`ALUResultM`)

La lógica prioriza los resultados de MEM sobre WB, seleccionando el valor más reciente cuando existen coincidencias. Además, se evita realizar forwarding desde escrituras cuyo destino es `x0`.

b) Stalling El stalling se utiliza cuando una instrucción depende de un dato que todavía no está disponible, como ocurre en una dependencia Load-Use.

La condición implementada en `hazard.v` es:

$$\begin{aligned} \texttt{lwStallD} = \texttt{ResultSrcEb0} \wedge \\ ((\texttt{Rs1D} = \texttt{RdE}) \vee (\texttt{Rs2D} = \texttt{RdE})) \end{aligned} \tag{2}$$

Cuando se detecta esta dependencia, se activan `StallF` y `StallD`, deteniendo el PC y conservando la instrucción en IF/ID. Simultáneamente, `FlushE` limpia ID/EX e introduce una burbuja.

Para ello, el datapath utiliza los módulos `flopenr`, `flopenrc` y `floprc`, que permiten habilitar o limpiar los registros correspondientes.

c) Flushing El flushing descarta instrucciones del camino incorrecto cuando se produce un salto tomado. En la implementación, los saltos se resuelven en Execute mediante:

$$\texttt{PCSrcE} = (\texttt{BranchE} \wedge \texttt{ZeroE}) \vee \texttt{JumpE} \tag{3}$$

Cuando `PCSrcE` se activa, la Hazard Unit genera `FlushD` y `FlushE`, anulando las instrucciones más jóvenes ubicadas en IF e ID. Asimismo, el PC se actualiza con la dirección objetivo del salto.

La señal `FlushE` también limpia las señales de control del registro ID/EX, evitando escrituras incorrectas en registros o memoria.

2.2. Diagrama del Datapath Single-Issue

El datapath Single-Issue integra las cinco etapas del pipeline, los registros intermedios IF/ID, ID/EX, EX/MEM y MEM/WB, así como las rutas de forwarding y las conexiones con la Hazard Unit.

La Figura 1 presenta el datapath de referencia con los multiplexores de selección de operandos y las señales que permiten detener o limpiar los registros del pipeline.

La Figura 2 muestra la integración de los módulos de la implementación. En `riscv.v` se conectan el datapath, el controlador y la Hazard Unit, mientras que `top.v` incorpora las memorias de instrucciones y datos.

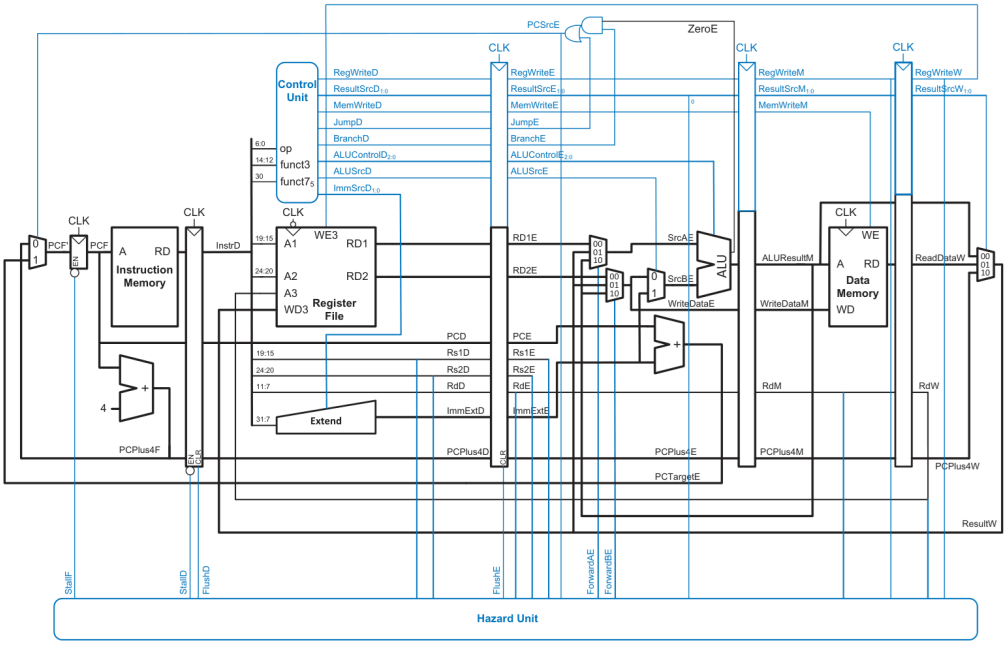


Figura 1: Datapath pipeline Single-Issue con mecanismos de resolución de hazards.

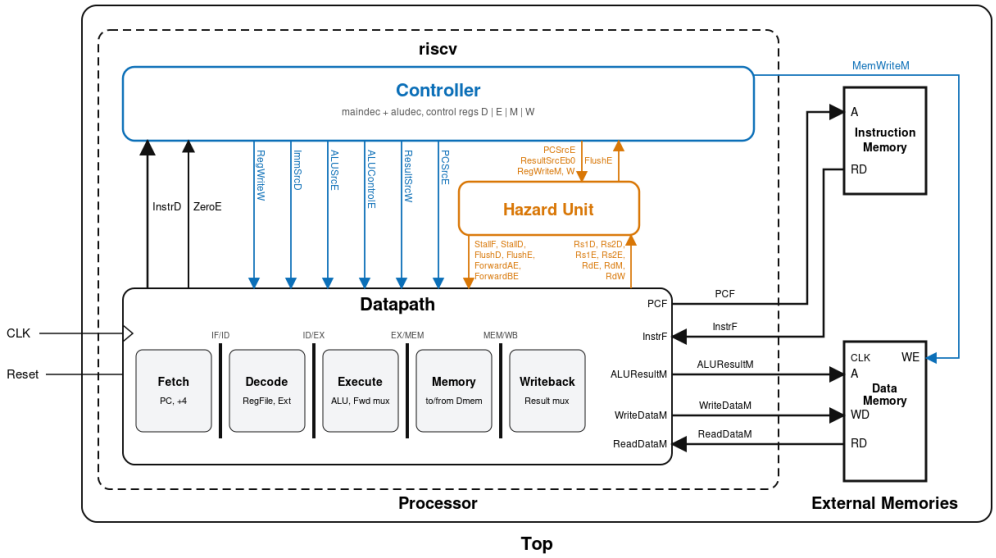


Figura 2: Integración de los módulos del procesador Pipeline Single-Issue.

3. Waveforms de los Cuatro Programas

3.1. Programa 1

El Programa 1 verifica el funcionamiento básico del pipeline de cinco etapas ejecutando las instrucciones soportadas por el procesador en una secuencia donde ninguna instrucción depende de otra a menos de tres posiciones. Con esta separación, el valor producido ya fue escrito en el banco de registros cuando se necesita (este escribe en el flanco de bajada del reloj), por lo que no se requiere forwarding, stalling ni flushing.

```
// Program 1 - ISA without dependencies that need the Hazard Unit
// Every consumer is 3+ instructions after its producer, so no
    forwarding/stall/flush.
// Expected stores: MEM[0]=17 [4]=7 [8]=8 [12]=17 [16]=1
00C00093  //    0: addi x1, x0, 12        ; x1 = 12
00500113  //    4: addi x2, x0, 5         ; x2 = 5
00A06193  //    8: ori x3, x0, 10         ; x3 = 10
00102393  //   12: slti x7, x0, 1         ; x7 = 1
00208233  //   16: add x4, x1, x2         ; 17
402082B3  //   20: sub x5, x1, x2         ; 7
0030F333  //   24: and x6, x1, x3         ; 8
00402023  //   28: sw x4, 0(x0)           ; MEM[0] = 17
00002483  //   32: lw x9, 0(x0)           ; x9 = 17
00502223  //   36: sw x5, 4(x0)           ; MEM[4] = 7
00602423  //   40: sw x6, 8(x0)           ; MEM[8] = 8
00902623  //   44: sw x9, 12(x0)          ; MEM[12] = 17 (from the lw)
00702823  //   48: sw x7, 16(x0)          ; MEM[16] = 1
0000006F  //   52: jal x0, 0              ; end: loop forever
```

Programa 1: ISA sin dependencias (`pipeline/isa.mem`).

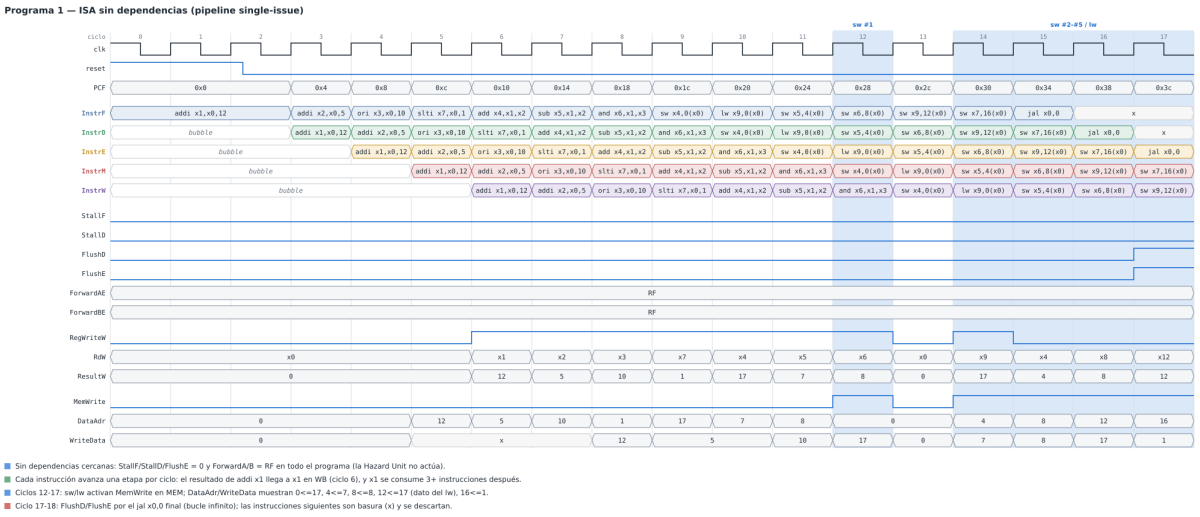


Figura 3: Waveform de la ejecución del Programa 1.

Análisis. En la Figura 3 se observa que la Hazard Unit no interviene mientras se

ejecutan las instrucciones del programa: `ForwardAE` y `ForwardBE` se quedan en RF (`00`), y `StallF` y `StallD` valen 0 en todo momento. `FlushD` y `FlushE` solo se activan en el ciclo 17, por el `jal` final. Esto pasa porque ninguna instrucción usa un registro que se calculó hace menos de tres instrucciones. Por ejemplo, `add x4,x1,x2` (dirección 16) necesita x1 y x2, que se calcularon en las direcciones 0 y 4. El valor de x1 ya está escrito cuando `add` llega a Decode (ciclo 7), y el de x2 se escribe en ese mismo ciclo, como se explica a continuación.

El caso más justo ocurre cuando el valor se calculó tres instrucciones antes. Ahí el dato llega a tiempo porque el Register File escribe en el flanco de bajada del reloj: la instrucción que lo produce está en WB mientras la que lo necesita está en Decode, y esta alcanza a leer el valor nuevo en la misma mitad de ciclo, sin necesitar forwarding. Se ve con `sw x4,0(x0)` (dirección 28), que usa el x4 calculado por `add x4` (dirección 16): en el ciclo 10, `add` está en WB (`RdW`=x4, `ResultW`=17) y `sw` está en Decode.

Las escrituras en memoria se ven cuando cada `sw` llega a la etapa MEM y `MemWrite` se pone en 1 (ciclos 12 y 14 a 17; en el ciclo 13 está el `lw` y baja a 0). El `lw x9,0(x0)` lee el 17 que se guardó en `MEM[0]`, y luego `sw x9,12(x0)` lo copia a `MEM[12]`, lo que confirma que la lectura de memoria y la escritura en WB funcionan bien. Al final, el `jal x0,0` deja el programa en un bucle infinito; en el ciclo 17, cuando se resuelve en EX, se activan `FlushD` y `FlushE` y las instrucciones que venían después se descartan.

Los resultados de memoria coinciden con los esperados: en la waveform, cada vez que `MemWrite` vale 1, `DataAdr` y `WriteData` muestran `MEM[0]` = 17, `MEM[4]` = 7, `MEM[8]` = 8, `MEM[12]` = 17 y `MEM[16]` = 1, que son los valores calculados a mano a partir del programa. Por lo tanto, el Programa 1 se ejecuta correctamente.

3.2. Programa 2

El Programa 2 verifica el *forwarding* entre etapas. A diferencia del Programa 1, aquí las instrucciones dependen de las inmediatamente anteriores (dependencias de distancia 1 y 2), por lo que el valor necesario aún no fue escrito en el banco de registros. La Hazard Unit debe reenviarlo directamente desde la etapa de Memory o de Writeback hacia la entrada de la ALU, sin detener el pipeline.

```
1  // Program 2 - Forwarding: back-to-back dependencies, no stalls
2  // Expected stores: MEM[0]=5 [4]=13 [8]=8
3  00500093  //   0: addi x1, x0, 5   ; x1 = 5
4  00308113  //   4: addi x2, x1, 3   ; x1 from MEM -> 8
5  001101B3  //   8: add  x3, x2, x1  ; x2 from MEM, x1 from WB -> 13
6  40218233  //  12: sub  x4, x3, x2  ; x3 from MEM, x2 from WB -> 5
7  00402023  //  16: sw   x4, 0(x0)   ; store data forwarded from MEM
8  00302223  //  20: sw   x3, 4(x0)
9  00202423  //  24: sw   x2, 8(x0)
10 0000006F  //  28: jal  x0, 0
```

Programa 2: Forwarding (`pipeline/forwarding.mem`).

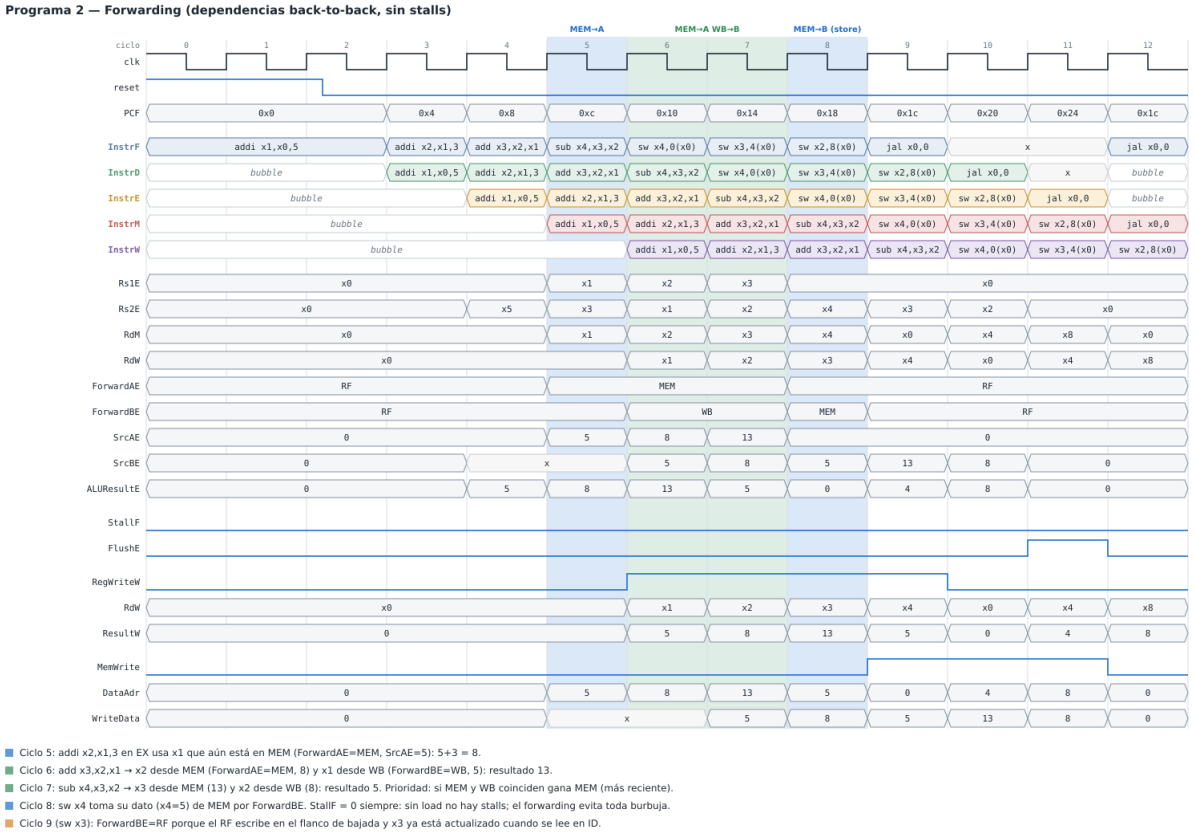


Figura 4: Waveform de la ejecución del Programa 2.

Análisis. En la Figura 4 cada instrucción necesita el resultado de la anterior. Ese resultado se calcula en la etapa EX, pero recién se escribe en el Register File (RF) cuando la instrucción llega a WB, dos etapas después. Mientras tanto, la instrucción siguiente ya está en Execute y necesita el valor. El *forwarding* resuelve esto enviando el resultado directamente desde la etapa MEM o WB a la entrada de la ALU, sin esperar a que se escriba en el RF y sin detener el pipeline.

En el ciclo 5, `addi x2,x1,3` toma x1 desde MEM (`ForwardAE`=MEM, `SrcAE`=5) y calcula $5 + 3 = 8$. En el ciclo 6, `add x3,x2,x1` toma x2 desde MEM (8) y x1 desde WB (5), y da 13. En el ciclo 7, `sub x4,x3,x2` toma x3 desde MEM (13) y x2 desde WB (8), y da 5. En el ciclo 8, `sw x4` recibe x4=5 desde MEM por `ForwardBE`. Los `sw x3` y `sw x2` de los ciclos 9 y 10 ya no necesitan forwarding (`ForwardBE`=RF), porque el RF escribe en el flanco de bajada y ya tiene esos valores.

Gracias al forwarding, `StallF` vale 0 en todo el programa y no hay burbujas. `FlushE` solo sube en el ciclo 11, por el `jal` final. Las escrituras en memoria (ciclos 9 a 11) son `MEM[0]` = 5, `MEM[4]` = 13 y `MEM[8]` = 8, iguales a los valores esperados, por lo que el Programa 2 se ejecuta correctamente.

3.3. Programa 3

El Programa 3 verifica el *stalling* ante un hazard de tipo load-use. Cuando una instrucción necesita el resultado de un `lw` que está inmediatamente antes, el dato recién está

disponible al final de la etapa Memory, por lo que el forwarding por sí solo no alcanza. La Hazard Unit detiene Fetch y Decode durante un ciclo e inserta una burbuja en Execute; después, el dato se reenvía desde Writeback.

```
// Program 3 - Stalling: load-use hazard (1 bubble)
// Expected stores: MEM[0]=42 [4]=43 [8]=86
02A00093  //   0: addi x1, x0, 42   ; x1 = 42
00102023  //   4: sw   x1, 0(x0)    ; MEM[0] = 42
00002103  //   8: lw   x2, 0(x0)    ; x2 = 42
00110193  //  12: addi x3, x2, 1    ; LOAD-USE: stall 1 cycle -> 43
00302223  //  16: sw   x3, 4(x0)    ; MEM[4] = 43
00402203  //  20: lw   x4, 4(x0)    ; x4 = 43
004202B3  //  24: add  x5, x4, x4   ; LOAD-USE on both operands:
    stall -> 86
00502423  //  28: sw   x5, 8(x0)    ; MEM[8] = 86
0000006F  //  32: jal  x0, 0
```

Programa 3: Stalling (`pipeline/stall.mem`).

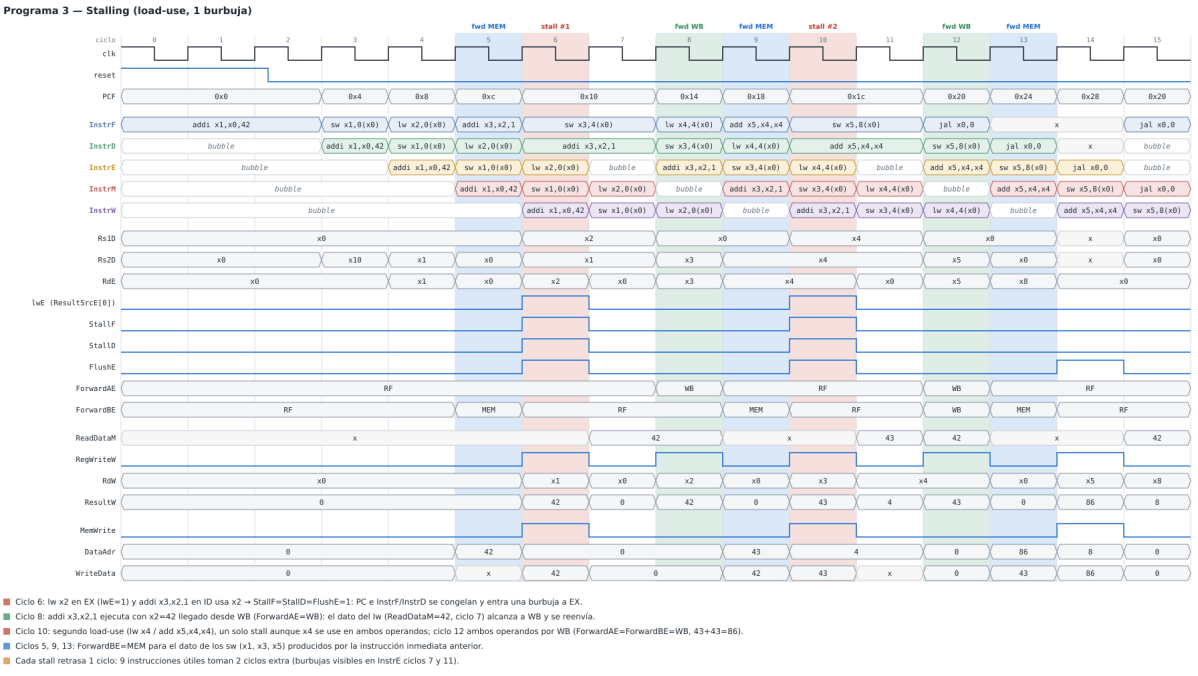


Figura 5: Waveform de la ejecución del Programa 3.

Análisis. En la Figura 5 hay dos casos de *load-use*: una instrucción necesita el valor que carga un `lw` que está justo antes. El forwarding no alcanza aquí, porque el dato de un `lw` recién se lee de la memoria en la etapa MEM, y la instrucción siguiente ya está en Execute y lo necesita en ese mismo momento. La solución es un *stall*: detener el pipeline un ciclo para que el dato llegue a tiempo. La Hazard Unit lo detecta cuando la instrucción en EX es un `lw` (`lwE`=1) y su registro destino (`RdE`) es igual a un registro fuente de la instrucción que está en Decode (`Rs1D` o `Rs2D`).

En el ciclo 6, `lw x2,0(x0)` está en EX y `addi x3,x2,1` está en Decode y usa x2 (`RdE`=`Rs1D`=x2). Entonces `StallF`, `StallD` y `FlushE` suben a 1: el PC (que se queda en

0x10) y la instrucción en Decode se congelan, y se inserta una burbuja en EX (InstrE muestra *bubble* en el ciclo 7). En ese ciclo el lw pasa a MEM y ReadDataM=42. En el ciclo 8, addi x3,x2,1 ya puede ejecutarse: el lw está en WB y ForwardAE=WB le envía x2=42, con lo que calcula $42 + 1 = 43$.

El segundo caso es el de lw x4,4(x0) seguido de add x5,x4,x4, que usa x4 en los dos operandos. Aun así, se necesita un solo stall (ciclo 10), porque ambos operandos dependen del mismo lw. En el ciclo 12, ForwardAE y ForwardBE valen WB, y la ALU calcula $43 + 43 = 86$. Los sw que van justo después de la instrucción que produce su dato (ciclos 5, 9 y 13) no necesitan stall, solo forwarding desde MEM (ForwardBE=MEM), como en el Programa 2.

Cada stall le cuesta un ciclo al programa, así que los dos stalls suman dos ciclos extra. Las escrituras en memoria se ven en los ciclos 6, 10 y 14 (MemWrite=1), con DataAdr/WriteData igual a MEM[0] = 42, MEM[4] = 43 y MEM[8] = 86. Coinciden con los valores esperados, por lo que el Programa 3 se ejecuta correctamente.

3.4. Programa 4

El Programa 4 prueba el *flushing*. Tiene un beq que se cumple y un jal, y después de cada salto hay dos instrucciones addi (con el valor 99) que pertenecen al camino equivocado y no deben ejecutarse. Si la Hazard Unit las descarta bien, el valor 99 nunca se escribe y en memoria queda el valor correcto.

```
1  // Program 4 - Flushing: taken beq and jal
2  // Wrong-path instructions would write 99; they must be flushed.
3  // Expected stores: MEM[0]=7 [4]=28
4  00700193  //   0: addi x3, x0, 7    ; x3 = 7 (must stay 7)
5  00500093  //   4: addi x1, x0, 5
6  00500113  //   8: addi x2, x0, 5
7  00208663  //  12: beq  x1, x2, 12   ; taken -> skips next 2
8  06300193  //  16: addi x3, x0, 99   ; flushed
9  06300193  //  20: addi x3, x0, 99   ; flushed
10 00C0036F  //  24: jal  x6, 12      ; x6 = 28, skips next 2
11 06300193  //  28: addi x3, x0, 99   ; flushed
12 06300193  //  32: addi x3, x0, 99   ; flushed
13 00302023  //  36: sw   x3, 0(x0)    ; still 7 -> wrong path flushed
14 00602223  //  40: sw   x6, 4(x0)    ; 28 (return address)
15 0000006F  //  44: jal  x0, 0
```

Programa 4: Flushing (pipeline/flush.mem).

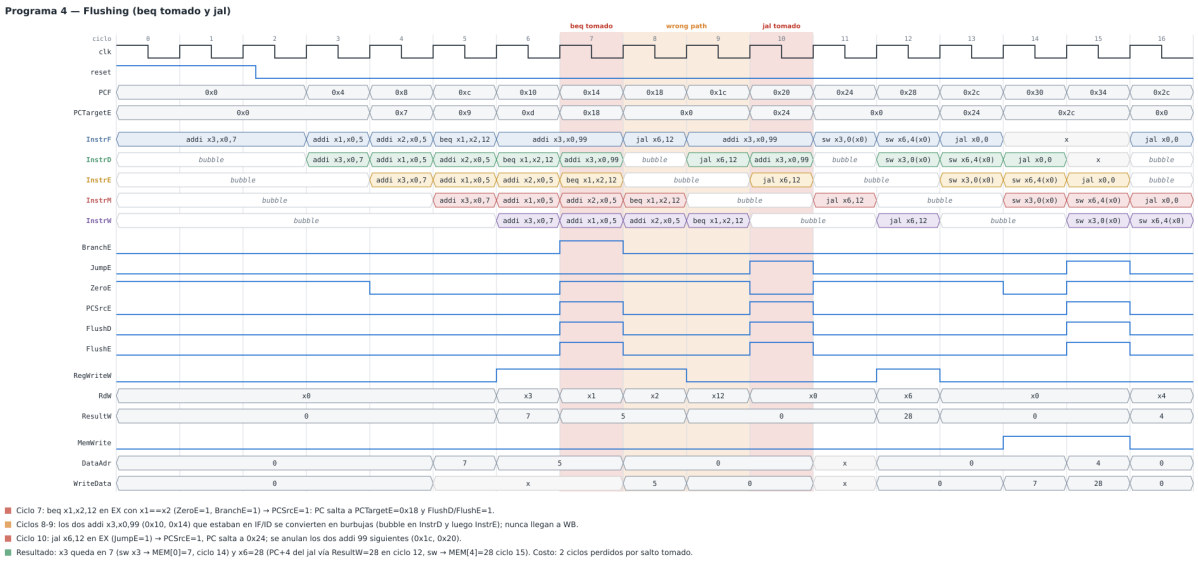


Figura 6: Waveform de la ejecución del Programa 4.

Análisis. En la Figura 6 se observa el *flushing*. El procesador decide si un salto se toma recién en la etapa Execute, y para ese momento ya buscó las dos instrucciones que siguen al salto en el programa. Si el salto se toma, esas dos instrucciones son del camino equivocado y hay que descartarlas, porque si llegaran a ejecutarse modificarían registros o memoria. La Hazard Unit lo hace con `FlushD` y `FlushE`, que se activan cuando `PCSrcE`=1 y borran los registros IF/ID e ID/EX, convirtiendo esas instrucciones en burbujas.

En el ciclo 7, `beq x1,x2,12` está en EX. En ese mismo ciclo el `beq` recibe `x1` desde WB (`ForwardAE`=WB) y `x2` desde MEM (`ForwardBE`=MEM), porque `x1` se calculó dos instrucciones antes y `x2` una antes, y ninguno está todavía en el Register File. Como x1 y x2 valen 5, la ALU da cero (`ZeroE`=1), y con `BranchE`=1 se activa `PCSrcE`. El PC salta a `PCTargetE`=`0x18`, y `FlushD` y `FlushE` valen 1. Los dos `addi x3,x0,99` que ya estaban en el pipeline (direcciones `0x10` y `0x14`) se convierten en burbujas en los ciclos 8 y 9 (*bubble* en `InstrD` e `InstrE`) y nunca llegan a WB. En el ciclo 10 ocurre lo mismo con `jal x6,12`: `JumpE`=1, `PCSrcE`=1, el PC salta a `0x24` y se descartan los dos `addi x3,x0,99` siguientes (`0x1c` y `0x20`).

El resultado confirma que el flush funcionó. Ninguna instrucción del camino equivocado llega a WB: en `ResultW` nunca aparece el 99. El primer `sw` escribe `MEM[0]` = 7 en el ciclo 14, es decir, el valor correcto y no 99. Además, `jal x6,12` guarda la dirección de retorno PC+4 = 28 (`RdW`=x6, `ResultW`=28 en el ciclo 12), y el segundo `sw` escribe `MEM[4]` = 28 en el ciclo 15. Ambos valores coinciden con los esperados.

3.5. Señales relevantes

Señal	Qué indica	Programa
`InstrF/D/E/M/W`	Instrucción que está en cada etapa del pipeline (solo para depurar)	1–4
`Rs1E`, `Rs2E`, `RdM`, `RdW`	Registros que compara la Hazard Unit para decidir el forwarding	2
`ForwardAE`, `ForwardBE`	Fuente de los operandos de la ALU: RF, WB o MEM	1–4
`SrcAE`, `SrcBE`, `ALUResultE`	Operandos que entran a la ALU y su resultado en Execute	2
`Rs1D`, `Rs2D`, `RdE`, `lwE`	Señales con las que se detecta el load-use: un `lw` en EX cuyo destino lo usa la instrucción en Decode	3
`StallF`, `StallD`	Congelan el PC y el registro IF/ID	1–3
`FlushD`, `FlushE`	Anulan las instrucciones en Decode y Execute (burbuja)	1–4
`BranchE`, `JumpE`, `ZeroE`	Salto condicional, salto incondicional y resultado cero de la ALU	4
`PCSrcE`, `PCTargetE`	Indica si se toma el salto y a qué dirección	4
`ReadDataM`	Dato que lee la memoria en la etapa MEM	3
`ResultW`, `RdW`	Valor y registro destino que se escriben en WB	1–4
`MemWrite`, `DataAdr`, `WriteData`	Escritura a memoria: cuándo, en qué dirección y con qué dato	1–4

Cuadro 1: Señales relevantes observadas en las waveforms.

3.6. Resultados intermedios

El Cuadro 2 resume lo observado en cada programa: el ciclo en que se hace el último `sw` y cuántas veces actuó la Hazard Unit (sin contar el `jal` final, que solo mantiene el bucle).

Programa	Último `sw`	Forwardings	Stalls	Flushes
1 (ISA)	ciclo 17	0	0	0
2 (Forwarding)	ciclo 11	6	0	0
3 (Stalling)	ciclo 14	6	2	0
4 (Flushing)	ciclo 15	2	0	2

Cuadro 2: Resumen de resultados por programa.

Algunos valores intermedios que se vieron en las waveforms: en el Programa 2, la ALU calculó 8, 13 y 5 (`ALUResultE`); en el Programa 3, el `lw` leyó 42 y 43 (`ReadDataM`) y la ALU calculó $42+1=43$ y $43+43=86$; en el Programa 4, `PCTargetE` valió `0x18` y `0x24`, y cada salto tomado descartó 2 instrucciones (2 ciclos perdidos).

4. Implementación Preliminar del Dual-Issue

La implementación preliminar Dual-Issue extiende el procesador Single-Issue mediante la obtención y decodificación paralela de dos instrucciones. En esta primera entrega, únicamente la vía 0 completa las etapas de ejecución, memoria y escritura. La segunda vía permanece preparada para su ampliación en la siguiente entrega.

4.1. Cambios en Fetch

Para leer dos instrucciones de 32 bits en el mismo ciclo, la etapa de *Fetch* tuvo estas modificaciones:

- **Memoria de instrucciones de doble lectura (`imem`):** Se modificó para entregar al mismo tiempo la instrucción de la dirección a y la siguiente, ubicada en $a + 4$:

$$rd1 = RAM[a[31:2]] \tag{4}$$
$$rd2 = RAM[a[31:2] + 1] \tag{5}$$

  Así, en cada ciclo se obtienen `InstrF_0` e `InstrF_1`.

- **Cálculo de PC+8:** Se agregó un sumador (`pcadd8`) que calcula $PC + 8$, y el multiplexor del PC (`pcmux`) pasó a tener tres entradas: $PC+4$, $PC+8$ y la dirección del salto. En esta entrega el selector solo elige $PC+4$ o la dirección del salto, porque únicamente la vía 0 ejecuta instrucciones; por eso el PC avanza de 4 en 4. La entrada $PC + 8$ queda lista para la segunda entrega, cuando la Issue Unit decida si emitir una o dos instrucciones por ciclo.

- **Registro IF/ID más ancho:** El registro entre Fetch y Decode (`regD`, de tipo `flopenrc`) pasó de 96 a 192 bits para guardar las dos instrucciones, sus PC y sus PC+4: `InstrF_0`, `InstrF_1`, `PCF`, `PCPlus4F` y `PCPlus8F`. Mantiene las señales `StallD` y `FlushD`.

4.2. Cambios en Decode

La etapa de *Decode* se modificó para decodificar las dos instrucciones del par en el mismo ciclo:

- **Decodificación paralela (`controller`):** Se instanciaron dos decodificadores idénticos (`maindec` y `aludec`), uno por cada vía. El de la vía 0 genera las señales de control de `InstrD_0` (`RegWriteD_0`, `ResultSrcD_0`, `MemWriteD_0`, `ALUSrcD_0`, `ALUControlD_0`, entre otras) y el de la vía 1 hace lo mismo con `InstrD_1`. Cada vía tiene además sus propios registros de control entre etapas.

- **Register File con 4 lecturas y 2 escrituras (`regfile`):** Para leer los operandos de ambas instrucciones a la vez, el Register File tiene cuatro puertos de lectura combinacionales y dos de escritura:

  - Vía 0: `a1` = `InstrD_0[19:15]` y `a2` = `InstrD_0[24:20]` (salidas `rd1`, `rd2`).
  - Vía 1: `a4` = `InstrD_1[19:15]` y `a5` = `InstrD_1[24:20]` (salidas `rd4`, `rd5`).

  - Escritura: puertos `we3/a3/wd3` y `we6/a6/wd6`, ambos en el flanco de bajada del reloj. Si las dos vías escriben el mismo registro, gana la vía 1 (la instrucción más joven). En esta entrega solo se usa el puerto de la vía 0; el de la vía 1 queda deshabilitado (`RegWriteW_1`= 0) hasta completar su etapa de Writeback.

- **Generadores de inmediato (`extend`):** Se instanciaron dos unidades de extensión de signo, `ext_0` y `ext_1`, que generan `ImmExtD_0` e `ImmExtD_1` en paralelo.

4.3. Módulos incorporados

Para pasar del pipeline Single-Issue a la versión Dual-Issue preliminar se modificaron los siguientes módulos:

- `imem.v`: memoria de instrucciones con dos salidas de 32 bits (`rd1` y `rd2`), que entregan la instrucción de la dirección a y la siguiente.
- `regfile.v`: Register File con 4 puertos de lectura y 2 de escritura.
- `controller.v`: tiene dos copias de los decodificadores (`maindec` y `aludec`) y registros de control separados para cada vía.
- `datapath.v`: incorpora el sumador `pcadd8`, el IF/ID de 192 bits, y la decodificación y el registro ID/EX de la vía 1. La ejecución, la memoria y el Writeback siguen solo en la vía 0.
- `hazard.v`: se amplió con los puertos de la vía 1, pero por ahora la lógica de forwarding, stall y flush solo se calcula para la vía 0; las salidas de forwarding de la vía 1 están fijas en `00`. La detección de dependencias entre las dos vías queda para la segunda entrega.
- `riscv.v` y `top.v`: conectan las señales de las dos vías entre el controller, el datapath, la Hazard Unit y la memoria de instrucciones (`InstrF_0` e `InstrF_1`).

El resto de los módulos (`alu`, `aludec`, `maindec`, `extend`, `dmem`, `mux2`, `mux3`, `adder` y los registros `flopr`, `floprc`, `flopenr` y `flopenrc`) se reutilizan sin cambios del pipeline Single-Issue.

4.4. Datapath Dual-Issue Preliminar

El datapath preliminar extiende el pipeline Single-Issue con una segunda vía (*Lane 1*) que, por ahora, llega hasta la etapa de Decode: se leen y decodifican dos instrucciones por ciclo, pero solo la vía 0 (*Lane 0*) ejecuta y escribe resultados. Esto permite verificar el Dual Fetch y el Dual Decode sin alterar el comportamiento ya validado con los cuatro programas.

- **Fetch (IF):** `imem` entrega al mismo tiempo dos instrucciones consecutivas (`InstrF_0` en la dirección `PCF` e `InstrF_1` en `PCF`+4). El PC avanza de 4 en 4 (`PCPlus4F`); el sumador `pcadd8` y la entrada $PC + 8$ del `pcmux` ya existen, pero se activarán en la segunda entrega.

- **Decode (ID):** el registro IF/ID (`regD`) tiene 192 bits y guarda las dos instrucciones, sus PC y sus PC+4. Cada instrucción tiene su propio decodificador de control, su generador de inmediato (`ext_0`, `ext_1`) y sus puertos de lectura del Register File (`rd1, rd2` y `rd4, rd5`). El registro ID/EX también está duplicado (`regE_0` y `regE_1`).
- **Execute (EX):** solo la vía 0 tiene ALU (`alu_0`), multiplexores de forwarding (`faemux_0` y `fbemux_0`) y sumador de saltos (`pcaddbranch_0`). La vía 1 no tiene todavía su ALU.
- **Memory y Writeback (MEM/WB):** la memoria de datos y la escritura en el Register File siguen asignadas a la vía 0 (`ALUResultM_0` y `WriteDataM_0`). El puerto de escritura de la vía 1 existe en el Register File (`we6`, `a6`, `wd6`), pero está deshabilitado (`RegWriteW_1`= 0).

4.5. Evidencia del Dual Fetch y Dual Decode

Para comprobar el Dual Fetch y el Dual Decode se ejecutó el Programa 1 (`pipeline/isa.mem`) sobre el pipeline Dual-Issue preliminar. La Figura 7 muestra la etapa Fetch, la decodificación de ambos carriles (vías) y el avance de las instrucciones por el pipeline.

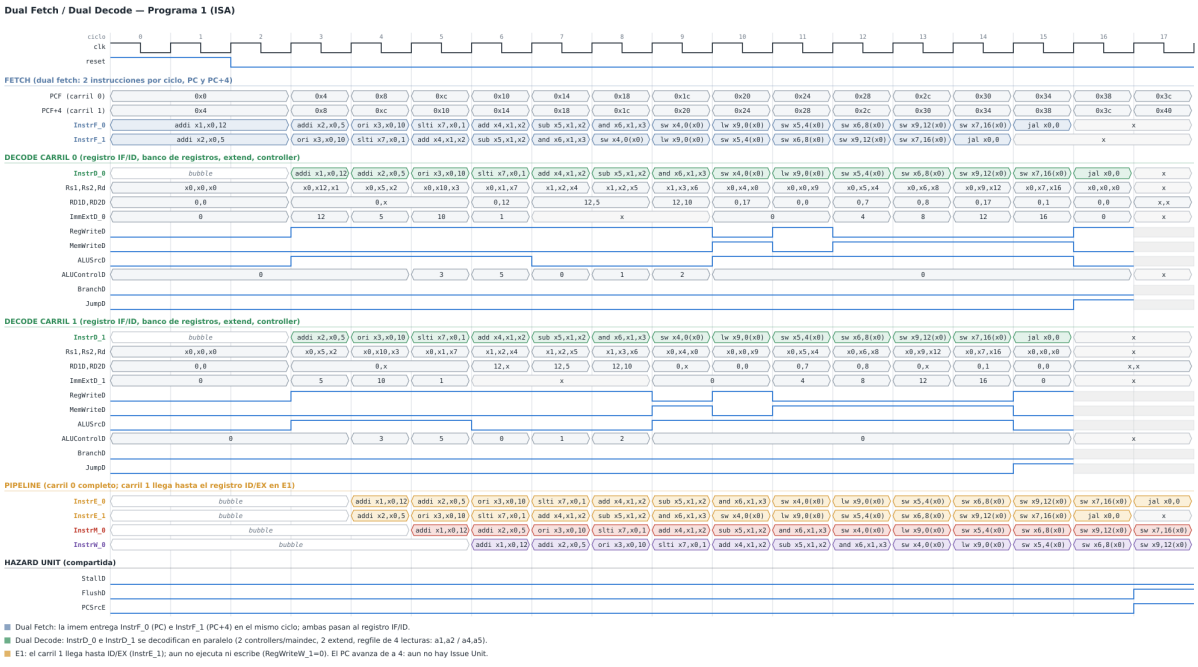


Figura 7: Waveform del Dual Fetch y Dual Decode (Programa 1).

Dual Fetch. En el ciclo 2, con `PCF` = `0x0`, la memoria de instrucciones entrega al mismo tiempo `InstrF_0` (`addi x1,x0,12`) e `InstrF_1` (`addi x2,x0,5`), que corresponden a las direcciones `0x0` y `0x4`. En el ciclo 3 el registro IF/ID ya contiene el par completo (`InstrD_0` e `InstrD_1`), por lo que ambas instrucciones se capturan en el mismo flanco.

Dual Decode. En cada ciclo la etapa Decode toma las dos instrucciones del registro IF/ID y las decodifica en paralelo: cada carril tiene su propio decodificador de control

(`maindec` y `aludec`), su generador de inmediato y sus puertos de lectura del Register File (`rd1`, `rd2` para el carril 0 y `rd4`, `rd5` para el carril 1). Esto se evidencia en la Figura 7. En el ciclo 7, `add x4,x1,x2` (carril 0) y `sub x5,x1,x2` (carril 1) leen los mismos registros por puertos distintos (`RD1D`,`RD2D` $= 12, 5$ en ambos carriles) y solo difieren en `ALUControlD`: 0 para `add` y 1 para `sub`. En el ciclo 9, el carril 0 decodifica `and x6,x1,x3` (tipo R, `ALUControlD` $= 2$) y el carril 1 decodifica `sw x4,0(x0)` (tipo S): el carril 1 activa `MemWriteD` y `ALUSrcD`, apaga `RegWriteD` y genera su inmediato (`ImmExtD_1` $= 0$), mientras que el carril 0 mantiene `RegWriteD` activo y `MemWriteD` y `ALUSrcD` en 0. Así, cada carril produce sus propias señales de control según la instrucción que recibe, sin depender del otro.